

7.1 方差分析与回归分析

7.1.1 方差分析

在统计学中，方差分析（ANOVA, Analysis of Variance）是一种用于比较两个或多个样本均值的统计方法，通过检验样本间是否存在显著性差异来判断这些样本是否来自同一总体。在Python中，statsmodels 库提供了进行方差分析的功能，主要使用 ols （普通最小二乘法）模型进行线性回归后，再应用方差分析的工具。然而，对于直接的方差分析，statsmodels 提供了 anova_lm 函数，该函数可以直接对线性模型的结果进行方差分析。

以下是一个如何使用 statsmodels 进行单因素方差分析（One-Way ANOVA）的基本示例：

1. **准备数据**：首先，你需要有一组数据，这些数据可以根据某个分类变量被分为不同的组。
2. **导入必要的库**：

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
from statsmodels.formula.api import ols
```

3. **构建数据**：这里我们使用 pandas 的 DataFrame 来模拟一些数据。

```
# 假设有三组数据，每组数据代表不同的处理方式
data = {'Group': ['A']*10 + ['B']*10 + ['C']*10,
        'Value': np.random.normal(0, 1, 10) + 1 + np.random.normal(0, 1, 10) + 2 + np.random.no
df = pd.DataFrame(data)
```

注意：这里为了演示效果，我们人为地给每组数据加上了不同的均值（1, 2, 3），并添加了随机噪声。

4. **使用 ols 模型并进行方差分析**：

虽然 ols 本身用于线性回归，但我们可以利用它来拟合数据，并通过 anova_lm 进行方差分析。

```
# 使用公式API定义模型，其中C(Group)表示将Group作为分类变量
model = ols('Value ~ C(Group)', data=df).fit()

# 使用anova_lm进行方差分析
anova_results = sm.stats.anova_lm(model, typ=1) # typ=1表示I型平方和

print(anova_results)
```

这里的 `C(Group)` 告诉 `ols` 将 `Group` 视为分类变量。 `anova_lm` 函数计算了ANOVA表，其中 `typ` 参数指定了平方和的类型（I型、II型或III型）。

5. 解读结果：

`anova_results` 将包含每个分类变量的F统计量、P值等信息。你可以根据P值来判断不同组之间是否存在显著性差异。如果P值小于显著性水平（通常是0.05），则认为这些组之间存在显著性差异。

请注意，以上示例是基于单因素方差分析（即只有一个分类变量）。如果你需要进行多因素方差分析（ANOVA with multiple factors），你可以在模型中加入更多的分类变量，并使用相同的流程进行分析。不过，对于复杂的实验设计，可能还需要考虑交互作用等因素。

7.1.2 回归分析

在Python中，使用 `statsmodels` 库进行线性回归是一种非常常见且强大的数据分析方法。下面我将给出一个简单的线性回归代码案例，用于展示如何使用 `statsmodels` 来拟合线性模型。

接下来是线性回归的代码案例：

```

import numpy as np
import statsmodels.api as sm
import pandas as pd

# 示例数据
# 假设我们有一组自变量X和因变量Y
X = np.array([1, 2, 3, 4, 5]) # 自变量
Y = np.array([2, 3, 5, 7, 11]) # 因变量, 这里  $Y = 1.4 * X + 0.5 + \text{noise}$ , 其中noise为随机噪声

# 为了使用statsmodels的OLS（普通最小二乘法）模型，我们需要将X转换为二维数组，
# 并添加一个常数项以拟合截距
X = sm.add_constant(X.reshape(-1, 1)) # 添加常数项，reshape(-1, 1)将X转换为二维数组

# 拟合模型
model = sm.OLS(Y, X).fit()

# 打印模型摘要
print(model.summary())

# 如果你想直接获取模型的参数（系数和截距），可以这样做：
print("模型参数: ", model.params)
# 输出中，第一个值是截距，第二个值是X的系数

# 使用模型进行预测
X_new = np.array([6, 7]).reshape(-1, 1) # 新数据点
X_new = sm.add_constant(X_new) # 同样需要添加常数项
predictions = model.predict(X_new)
print("预测值: ", predictions)

```

在这个例子中，我们首先生成了一组简单的自变量 x 和因变量 y 的数据，其中 y 是 x 的线性函数加上一些随机噪声。然后，我们使用 statsmodels 的 OLS 类来拟合这些数据。在拟合之前，我们通过在 x 中添加一个常数项来准备数据，这是为了模型能够拟合出截距项。

`fit()` 方法用于拟合模型，而 `summary()` 方法则提供了一个模型的详细摘要，包括模型的R-squared值、每个系数的统计量和p值等重要信息。

最后，我们使用拟合好的模型对新的数据点 `X_new` 进行预测，并打印出预测结果。

请注意，为了使用 `predict()` 方法进行预测，新数据点 `X_new` 也需要被转换为二维数组，并且同样需要添加常数项。

在 statsmodels 中，广义线性模型（Generalized Linear Models, GLM）是一种灵活的统计模型，用于估计因变量的条件均值，该因变量与自变量之间的关系通过链接函数（link function）和指数族分布（exponential family distributions）来指定。GLM 扩展了普通线性模型（OLS），允许因变量遵循非正态分布（如二项分布、泊松分布等），并且因变量的期望值可以通过非线性链接函数与自变量线性相关。

以下是如何使用 statsmodels 进行广义线性模型（GLM）的一个基本示例：

首先，确保你已经安装了 statsmodels 库。如果没有安装，可以通过pip安装：

```
pip install statsmodels
```

然后，你可以使用以下代码来拟合一个GLM：

```

import numpy as np
import statsmodels.api as sm

# 示例数据
# 自变量X和因变量Y（这里Y是二项分布的数据，例如成功/失败的次数）
X = np.random.rand(100, 2) # 生成100个样本，每个样本2个特征
# 假设真实的模型是  $\text{logit}(p) = 0.5 + 1.5 \cdot X[:,0] - 0.8 \cdot X[:,1]$ ，其中p是成功的概率
p = 1 / (1 + np.exp(-(0.5 + 1.5*X[:,0] - 0.8*X[:,1])))
Y = np.random.binomial(n=1, p=p) # 根据p生成二项分布的因变量

# 为了使用GLM，我们需要将Y转换为“成功”的次数（在这里，成功就是Y=1的情况）
# 对于二项分布，我们还需要知道每次试验的次数（这里是1），但GLM在statsmodels中通常只接受成功次数
# 注意：对于二项GLM，我们实际上只需要Y（成功次数）和n（试验次数）的数组，但statsmodels的GLM接口只接受
# 如果n不是固定的，则需要以不同的方式处理（例如，使用权重或转换为其他形式的数据）

# 添加常数项以拟合截距
X = sm.add_constant(X)

# 拟合GLM模型，这里使用logit链接函数和二项分布
model = sm.GLM(Y, X, family=sm.families.Binomial())
result = model.fit()

# 打印模型摘要
print(result.summary())

# 预测新数据点的概率
X_new = np.array([[0.2, 0.3], [0.8, 0.1]])
X_new = sm.add_constant(X_new)
predicted_probs = result.predict(X_new, linear=False) # linear=False给出预测的概率，而非线性预测
print("预测的成功概率: ", predicted_probs)

```

在这个例子中，我们生成了一组模拟数据，其中因变量 Y 是二项分布的，表示成功或失败的次数（在这个案例中，试验次数 n 固定为1）。然后，我们使用 `statsmodels` 的 `GLM` 类来拟合一个GLM模型，其中指定了 `Binomial` 分布和 `logit` 链接函数。拟合完成后，我们打印了模型的摘要，并使用拟合好的模型来预测新数据点的成功概率。

注意，在调用 `predict` 方法时，`linear=False` 参数表示我们想要得到的是通过链接函数反变换后的预测概率，而不是线性预测值。如果 `linear=True`（默认值），则 `predict` 方法将返回线性预测值，这些值需要通过链接函数的反函数来转换为概率或其他适当的度量。

7.1.3 方差分析与回归分析是统一的框架

线性回归和方差分析 (Analysis of Variance, ANOVA) 虽然在统计方法和应用场景上有所不同, 但它们在某些方面共享着相似的逻辑和框架, 尤其是在处理变量之间的关系和差异时。它们各自有独特的目标、假设和适用场景。不过, 可以从以下几个方面来理解它们之间的联系:

1. 模型基础

- i. **线性关系**: 线性回归是基于自变量 (解释变量) 和因变量之间线性关系的假设, 通过最小二乘法等方法来拟合这种关系。方差分析虽然不直接建立自变量和因变量之间的线性模型, 但它也隐含了变量之间可能存在的某种关系或差异, 这种关系或差异可以通过比较不同组之间的均值来揭示。
- ii. **统计推断**: 两者都涉及统计推断, 即利用样本数据来推断总体特征。线性回归通过估计回归系数和进行假设检验来推断自变量对因变量的影响; 方差分析则通过比较不同组之间的均值差异来推断某个因素 (自变量) 对观测值 (因变量) 的影响是否显著。

2. 假设条件

- i. **独立性**: 两者都要求观测值之间相互独立, 即一个观测值不影响其他观测值。
- ii. **正态性**: 虽然线性回归对误差项的正态性有明确要求 (误差项服从均值为0的正态分布), 但方差分析在某些情况下 (如单因素方差分析) 对原始数据的正态性要求不是非常严格, 更关注于组间差异的显著性。然而, 在更复杂的方差分析 (如多因素方差分析) 中, 也会考虑正态性和同方差性等假设条件。

3. 应用场景

- i. **线性回归**: 主要用于预测和解释, 通过建立自变量和因变量之间的线性关系模型, 可以预测因变量的值或解释自变量对因变量的影响程度。
- ii. **方差分析**: 主要用于比较和推断, 通过比较不同组之间的均值差异来推断某个因素 (自变量) 对观测值 (因变量) 的影响是否显著。它常用于实验设计、社会科学研究等领域。

4. 结合使用

在某些情况下, 线性回归和方差分析可以结合使用以提供更全面的数据分析结果。例如, 在方差分析得到群体均值差异显著的情况下, 可以进一步使用线性回归来确定差异的原因和自变量与因变量之间的具体关系。